TP Contours – Comparaison de trois détecteurs de contours

Sobel, Laplace et Canny évalués sur la base de contours de l'université de Groningen

40 – OpenCV – Homework 3 · Luis Eyraud · Octobre 2026 · Code source : `homework3.cpp`

En bref. Sur les 40 images de la base, avec le meilleur réglage de chaque détecteur, la performance moyenne vaut P = 0,407 pour Sobel, 0,325 pour Canny et 0,236 pour Laplace. Ce classement favorise Sobel, parce que ses contours épais sont comptés plusieurs fois comme corrects. Pourtant, Canny fait à la fois moins de faux positifs et moins de faux négatifs que Sobel, et il devient le meilleur détecteur dès que chaque pixel de référence retrouvé n'est compté qu'une fois (P' = 0,341 contre 0,260 pour Sobel). Le Laplacien est le plus sensible au bruit et à la texture. Pour les trois détecteurs, les erreurs viennent surtout des textures (herbe, feuillage, pelage), que la référence ne contient pas.

1. Objectif et démarche

L'objectif est de comparer trois détecteurs de contours (Sobel, Laplace, Canny) de façon quantitative, en confrontant les contours qu'ils produisent à des contours de référence tracés à la main. Les images utilisées sont celles de la base *contour database* de l'université de Groningen (RuG) : 40 images en niveaux de gris de 512 × 512 pixels (animaux et objets dans des scènes naturelles), chacune accompagnée d'une image binaire de référence où seuls les contours des objets ont été dessinés.

Le programme `homework3.cpp` permet de choisir le détecteur, de régler ses paramètres et de traiter le lot complet d'images en calculant, pour chaque image, cinq grandeurs (contours détectés, contours de référence, contours corrects, faux positifs, faux négatifs) et trois mesures : la performance P, le taux de faux positifs TFP et le taux de faux négatifs TFN. Pour comparer les détecteurs dans de bonnes conditions, chaque détecteur a été évalué avec le jeu de paramètres qui maximise sa performance moyenne sur les 40 images.

2. Présentation théorique des détecteurs

2.1 Contour, gradient et dérivée seconde

Un contour correspond à une variation brusque de l'intensité de l'image. Sur un profil d'intensité $I(x)$ qui passe d'une région sombre à une région claire, la dérivée première présente un **maximum** à l'endroit du contour, et la dérivée seconde y change de signe : elle présente un **passage par zéro**. Les détecteurs de type gradient (Sobel, Canny) cherchent les maxima de la dérivée première ; le détecteur de Laplace cherche les passages par zéro de la dérivée seconde.

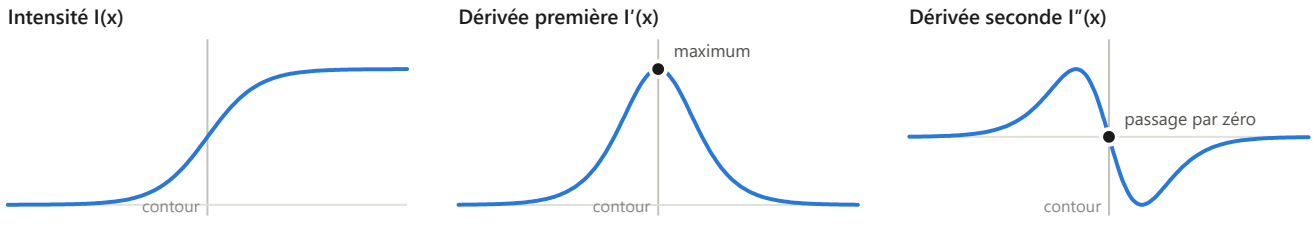


Figure 1 – Profil d'un contour (passage d'une zone sombre à une zone claire) : la dérivée première est maximale et la dérivée seconde s'annule à l'emplacement du contour.

En 2D, la dérivée première est le vecteur gradient $\nabla I = (G_x, G_y)$. Sa norme mesure la force du contour et son orientation $\theta = \text{atan2}(G_y, G_x)$ est perpendiculaire au contour. Les dérivées amplifiant le bruit, elles sont toujours associées à un lissage.

2.2 Détecteur de Sobel

Le filtre de Sobel approxime les dérivées partielles par une convolution avec deux noyaux 3 × 3. Chaque noyau combine une dérivée centrée [−1 0 1] dans une direction et un lissage [1 2 1] dans la direction perpendiculaire, ce qui le rend un peu moins sensible au bruit qu'une simple différence de pixels voisins :

$$G_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix} * I \qquad G_y = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix} * I$$

On calcule ensuite la norme du gradient $|\nabla I| = \sqrt{(G_x^2 + G_y^2)}$, puis on la **seuille** : un pixel est un contour si la norme dépasse le seuil. Les noyaux de taille 5 ou 7 lissent davantage. Le détecteur est simple et rapide, mais il ne garde pas uniquement le maximum : un contour flou donne une bande de plusieurs pixels au-dessus du seuil, donc des **contours épais**. Le seuil unique impose aussi un compromis : un seuil bas garde le bruit et la texture, un seuil haut coupe les contours faibles.

2.3 Détecteur de Laplace

Le laplacien est la somme des dérivées secondes : $\nabla^2 I = \partial^2 I/\partial x^2 + \partial^2 I/\partial y^2$. C'est un opérateur isotrope (une seule convolution, sans direction privilégiée) dont le noyau 3 × 3 le plus simple est :

$$\nabla^2 \approx \begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

Les contours sont localisés aux **passages par zéro** du laplacien, ce qui donne des contours fins (1 pixel) et fermés. La dérivée seconde est cependant très sensible au bruit : on applique d'abord un lissage gaussien (c'est le principe du *Laplacien de Gaussienne*, ou détecteur de Marr-Hildreth). Même après lissage, chaque petite variation d'intensité produit un passage par zéro. On ne garde donc que les passages par zéro où le laplacien change fortement de valeur, c'est-à-dire où la **pente** est supérieure à un seuil.

2.4 Détecteur de Canny

Canny (1986) a défini le détecteur à partir de trois critères : une **bonne détection** (peu de contours manqués et peu de faux contours), une **bonne localisation** (le contour détecté est au plus près du contour réel) et une **réponse unique** (un seul pixel de large par contour). L'algorithme comporte quatre étapes :

- **Lissage gaussien** pour réduire le bruit (écart-type σ) ;
- **Calcul du gradient** (norme et orientation), en pratique avec les filtres de Sobel ;
- **Suppression des non-maxima** : un pixel n'est gardé que si sa norme est maximale dans la direction du gradient, ce qui amincit les contours à un pixel ;
- **Seuillage par hystérésis** avec deux seuils : les pixels au-dessus du seuil haut sont des contours sûrs ; ceux entre les deux seuils ne sont gardés que s'ils sont connectés à un contour sûr. On garde ainsi les contours faibles qui prolongent un contour fort, sans garder les pixels faibles isolés. Canny recommande un rapport de 2 à 3 entre le seuil haut et le seuil bas.

3. Fonctions OpenCV utilisées et lien avec les paramètres

Le tableau suivant détaille les fonctions utilisées dans `homework3.cpp`, et la façon dont leurs paramètres correspondent aux éléments théoriques de la partie 2. Les noms en gras sont ceux des curseurs du programme.

Fonction	Rôle et paramètres	Lien avec la théorie
`cv::GaussianBlur(src, dst, cv::Size(), sigma)`	Lissage gaussien appliqué avant chaque détecteur. σ = **Flou** / 2 (Flou = 0 : pas de lissage). La taille du noyau est déduite de σ par OpenCV.	Réduit le bruit avant la dérivation. C'est le « G » du Laplacien de Gaussienne et la première étape de Canny (`cv::Canny` ne lisse pas lui-même).
`cv::Sobel(src, dst, CV_32F, dx, dy, ksize)`	`(1, 0)` donne G_x, `(0, 1)` donne G_y. `ksize` = 2 × **Noyau** + 1 ∈ {1, 3, 5, 7}. Le type `CV_32F` conserve les valeurs négatives.	Noyaux de Sobel de la partie 2.2. `ksize` = 1 donne la dérivée [−1 0 1] sans lissage ; 5 et 7 lissent davantage.
`cv::cartToPolar(gx, gy, norme, orientation)`	Calcule la norme $\sqrt{(G_x^2 + G_y^2)}$ et l'orientation du gradient (méthode de la classe `EdgeDetector` de la correction).	Force du contour $\lvert\nabla I\rvert$.
`cv::minMaxLoc` + `convertTo(dst, CV_8U, 255 / max)`	Ramène la norme (ou la pente du laplacien) dans [0, 255] par rapport à son maximum dans l'image.	Le **Seuil** de Sobel et de Laplace est donc relatif : 50 signifie « 50/255 ≈ 20 % du contour le plus fort de l'image ».
`cv::threshold(src, dst, seuil, 255, cv::THRESH_BINARY)`	Binarisation : 255 si la valeur dépasse **Seuil**.	Seuillage de la norme du gradient (Sobel) ou de la pente (Laplace).
`cv::Laplacian(src, dst, CV_32F, ksize)`	`ksize` = 2 × **Noyau** + 1. Pour `ksize` = 1, OpenCV utilise le noyau 3 × 3 de la partie 2.3 ; au-delà, la somme des dérivées secondes de Sobel de cette taille.	Dérivée seconde $\nabla^2 I$.
`cv::threshold(lap, signe, 0, …)`, `cv::dilate`, soustraction	Image du signe du laplacien, dilatée, moins l'image du signe : il reste les pixels négatifs qui touchent un pixel positif (méthode `getZeroCrossings` de la classe `LaplacianZC` de la correction).	Passages par zéro du laplacien.
`cv::morphologyEx(lap, pente, cv::MORPH_GRADIENT, cv::Mat())` + `cv::bitwise_and`	Gradient morphologique : maximum moins minimum du laplacien dans un voisinage 3 × 3. Cette pente est normalisée, seuillée par **Seuil**, puis combinée par un ET logique avec les passages par zéro.	Ne garde que les passages par zéro « francs », qui correspondent à un vrai saut d'intensité.
`cv::Canny(src, dst, seuil, seuilHaut)`	`threshold1` = **Seuil** (seuil bas), `threshold2` = **Seuil haut**. Valeurs par défaut : `apertureSize` = 3 (Sobel 3 × 3) et `L2gradient` = `false` (norme $\lvert G_x\rvert + \lvert G_y\rvert$).	Étapes 2 à 4 de Canny : gradient, suppression des non-maxima et hystérésis. Les seuils sont absolus. La taille de noyau reste à 3, car avec 5 ou 7 la norme du gradient change d'échelle et les seuils n'ont plus le même sens.
`cv::dilate(src, dst, cv::Mat())`, `cv::bitwise_and`, `cv::countNonZero`	Calcul des mesures. L'élément structurant par défaut est un carré 3 × 3, ce qui donne le voisinage de tolérance demandé.	Comparaison avec la référence (partie 4.2).

4. Le programme

4.1 Interface et traitement du lot

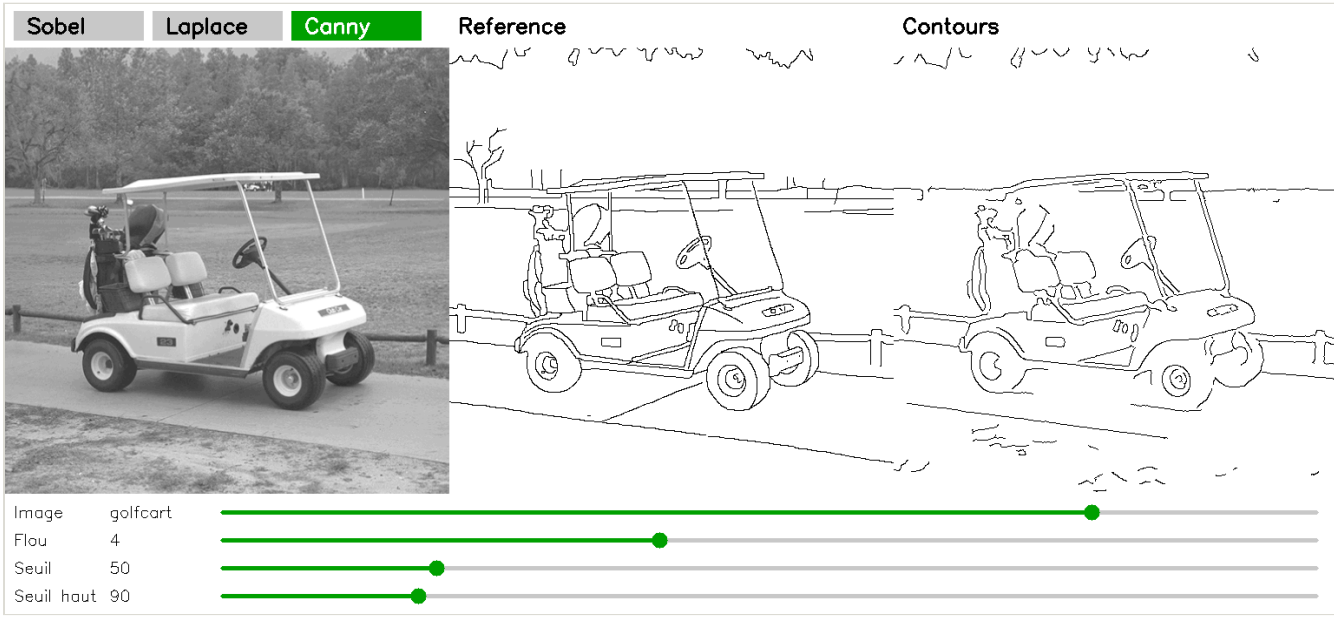


Figure 2 – Fenêtre unique du programme : boutons de choix du détecteur, l'image, sa référence et les contours détectés, puis les curseurs des paramètres du détecteur choisi (ici Canny sur golfcart : Image, Flou, Seuil et Seuil haut ; pour Sobel et Laplace, le curseur Seuil haut est remplacé par Noyau).

Au lancement, le programme charge toutes les images `*.pgm` du dossier donné en argument (par défaut `images`) avec `cv::glob`, ainsi que leur référence `gt/<nom>_gt_binary.pgm`. Les contours de référence sont noirs sur fond blanc : ils sont inversés par `cv::threshold(…, cv::THRESH_BINARY_INV)` pour que les pixels de contour valent 255, comme dans les images calculées.

Une seule fenêtre affiche l'image, sa référence et les contours détectés. Le détecteur se choisit en cliquant sur les boutons **Sobel**, **Laplace** et **Canny** (dessinés avec `cv::rectangle` et `cv::putText`, clic géré par `cv::setMouseCallback`). Sous les images, des curseurs règlent les paramètres : **Image** et **Flou** pour tous les détecteurs, **Noyau** et **Seuil** pour Sobel et Laplace, **Seuil** (seuil bas) et **Seuil haut** pour Canny. Seuls les curseurs utilisés par le détecteur choisi sont affichés. Les trackbars d'OpenCV ne pouvant pas être masquées, ces curseurs sont dessinés dans l'image (`cv::line`, `cv::circle`, `cv::putText`) et déplacés à la souris (événements `EVENT_LBUTTONDOWN`, `EVENT_MOUSEMOVE` et `EVENT_LBUTTONUP`). À chaque modification, les mesures de l'image courante s'affichent dans la console. La touche `l` traite tout le lot avec les paramètres courants : le programme affiche les 5 grandeurs et les 3 mesures de chaque image, puis la moyenne de *P*, *TFP* et *TFN*.

4.2 Calcul des mesures

Comme le conseille la consigne, un pixel détecté est considéré comme **correct** s'il existe un pixel de contour de référence dans son voisinage 3 × 3. En pratique, on dilate la référence avec un élément 3 × 3, puis on fait un ET logique avec les contours détectés :

- **contours_détectés** = nombre de pixels non nuls de l'image calculée ;
- **contours_référence** = nombre de pixels non nuls de la référence ;
- **contours_corrects** = détectés ∩ dilatation(référence) ;
- **faux_positifs** = contours_détectés − contours_corrects ;
- **faux_négatifs** = pixels de référence qui n'ont *aucun* pixel détecté dans leur voisinage 3 × 3, soit référence ∩ non(dilatation(détectés)).

Pour les faux négatifs, la formule « contours_référence − contours_corrects » n'est exacte que sans voisinage. Avec un voisinage, un contour détecté sur 3 pixels d'épaisseur compte 3 pixels corrects pour 1 pixel de référence : le nombre de contours corrects peut alors dépasser le nombre de contours de référence, et la soustraction devient négative. Avec le réglage retenu pour Sobel, c'est le cas pour 18 images sur 40 : pour l'image gnu, on obtient 10 836 pixels corrects pour 6 056 pixels de référence, soit « 6 056 − 10 836 = −4 780 » faux négatifs, alors que 1 313 pixels de référence n'ont réellement aucun pixel détecté dans leur voisinage. On applique donc la même tolérance dans l'autre sens, ce qui correspond à la définition littérale (« pixels contours non détectés, mais contours dans la référence »). Les trois mesures sont ensuite :

$$P = C / (C + FP + FN) \qquad TFP = FP / (C + FP + FN) \qquad TFN = FN / (C + FP + FN)$$

avec C = contours_corrects. On a toujours $P + TFP + TFN = 1$: un détecteur parfait obtient $P = 1$.

4.3 Choix des paramètres

Les curseurs permettent de voir l'effet de chaque paramètre. Pour comparer les détecteurs à leur meilleur niveau, un petit programme de test (`rapport/balayage.cpp`), qui réutilise les fonctions `detecterContours` et `comparer` de `homework3.cpp`, a évalué toutes les combinaisons de la grille suivante sur les 40 images :

- **Sobel et Laplace** : Flou de 0 à 8 (σ de 0 à 4), taille de noyau 1, 3, 5 ou 7 (Noyau de 0 à 3), Seuil de 5 à 200 par pas de 5 (1 440 combinaisons chacun) ;
- **Canny** : Flou de 0 à 8, Seuil bas de 10 à 300 par pas de 10, Seuil haut du seuil bas à 500 par pas de 20 (4 860 combinaisons).

Pour chaque détecteur, on retient le réglage qui maximise la performance moyenne P sur les 40 images. Le même réglage est utilisé pour toutes les images, comme lors d'un traitement par lot.

5. Résultats

5.1 Tableau des mesures

Détecteur	Réglage retenu	Détectés	Référence	Corrects	Faux positifs	Faux négatifs	*P*	*TFP*	*TFN*
Sobel	**Flou 3 (σ = 1,5) noyau 3 × 3, seuil 75**	**418 095**	**229 597**	**219 734**	**198 361**	**121 036**	**0,407**	**0,351**	**0,242**
Laplace	Flou 5 (σ = 2,5) noyau 7 × 7, seuil 50	350 459	229 597	102 210	248 249	120 916	0,236	0,481	0,284
Canny	Flou 4 (σ = 2,0) seuils 50 / 90	246 089	229 597	111 916	134 173	108 819	0,325	0,344	0,331

Tableau 1 – Résultats sur le lot de 40 images avec le meilleur réglage de chaque détecteur. Les cinq grandeurs sont sommées sur les 40 images ; *P*, *TFP* et *TFN* sont les moyennes des valeurs calculées image par image (comme dans la ligne « moyenne » du programme). σ = Flou / 2 ; un noyau $k \times k$ correspond au curseur Noyau = $(k - 1) / 2$. Le détail par image est en annexe.

Avec la mesure demandée, **Sobel obtient la meilleure performance moyenne** (P = 0,407), devant Canny (0,325) et Laplace (0,236). Les cinq grandeurs nuancent ce classement :

- Sobel détecte 418 095 pixels pour 229 597 pixels de référence, soit 1,82 fois plus. Canny en détecte 246 089, un nombre proche de celui de la référence (1,07 fois).
- Canny a **le moins de faux négatifs** (108 819) : il retrouve 52,6 % des pixels de référence, contre 47,3 % pour Sobel et 47,3 % pour Laplace.
- Canny a aussi **le moins de faux positifs** (134 173, contre 198 361 pour Sobel et 248 249 pour Laplace).
- Sobel a pourtant presque deux fois plus de pixels corrects que Canny (219 734 contre 111 916) : ses contours font plusieurs pixels d'épaisseur, et chacun de ces pixels compte comme correct s'il est à moins d'un pixel de la référence. C'est ce qui fait monter son P (voir partie 5.4).

La performance varie beaucoup d'une image à l'autre : pour Canny, de 0,104 (bear_7) à 0,517 (hyena). Les images difficiles (ours, chèvres) ont un fond très texturé, alors que la référence ne contient que le contour des objets. Enfin, le même réglage est utilisé pour toutes les images. En choisissant le meilleur réglage *image par image*, la performance moyenne monterait à 0,461 pour Sobel, 0,391 pour Canny et 0,276 pour Laplace : comme l'annonçait la consigne, un seul jeu de paramètres n'est pas adapté à toutes les images.

5.2 Influence des paramètres

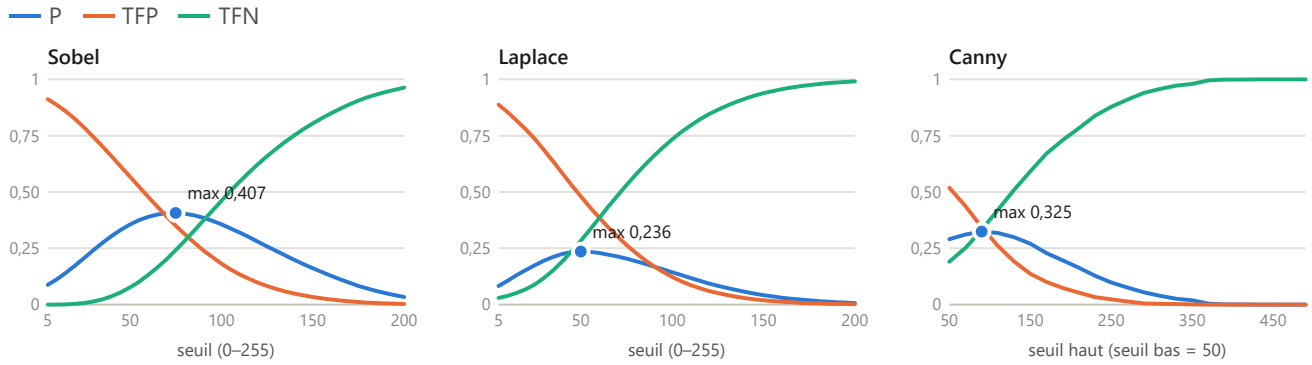


Figure 3 – Performance *P*, taux de faux positifs *TFP* et de faux négatifs *TFN* (moyennes sur les 40 images) en fonction du seuil, les autres paramètres étant fixés à leur meilleur réglage. Le point marque le réglage retenu.

Flou (σ)	Sobel	Laplace	Canny
0 (0,0)	0,397	0,192	0,266
1 (0,5)	0,401	0,195	0,279
2 (1,0)	0,407	0,207	0,310
3 (1,5)	**0,407**	0,220	0,325
4 (2,0)	0,399	0,232	**0,325**
5 (2,5)	0,381	**0,236**	0,314
6 (3,0)	0,363	0,234	0,298
7 (3,5)	0,344	0,228	0,281
8 (4,0)	0,324	0,219	0,263

Tableau 2 – Meilleure performance moyenne *P* atteignable pour chaque valeur du flou (les autres paramètres étant optimisés). En gras : le maximum de chaque détecteur.



σ = 0,0 – *P* = 0,222, *TFP* = 0,768, *TFN* = 0,010 | σ = 2,0 – *P* = 0,490, *TFP* = 0,128, *TFN* = 0,382 | σ = 4,0 – *P* = 0,116, *TFP* = 0,016, *TFN* = 0,868

Figure 4 – Effet du flou sur Canny (image golfcart, seuils 50 / 90) : sans lissage, la texture de l'herbe et des arbres est détectée ; avec trop de lissage, seuls les contours les plus forts subsistent.

Seuils. Pour les trois détecteurs, augmenter le seuil fait baisser le taux de faux positifs et monter le taux de faux négatifs (figure 3) : la performance est maximale au point d'équilibre entre ces deux erreurs. Le seuil optimal de Sobel vaut 75/255, soit environ 29 % de la norme maximale de chaque image. Celui de Laplace vaut 50/255. Pour Canny, le meilleur couple est 50 / 90 (rapport 1,8). Ces valeurs sont basses parce que le lissage (σ = 2,0) réduit la norme du gradient : les seuils de Canny sont absolus et doivent être réglés à nouveau quand on change le flou. Le seuil haut a un effet plus fort que le seuil bas, car c'est lui qui décide quels contours sont « sûrs ».

Flou. Les trois détecteurs profitent d'un lissage modéré (tableau 2), mais pas dans la même mesure. Sobel gagne peu (0,397 sans flou, 0,407 au mieux), car son noyau lisse déjà dans une direction. Canny gagne beaucoup (0,266 → 0,325) : le lissage supprime une grande partie de la texture avant la suppression des non-maxima. Laplace a besoin du lissage le plus fort (σ = 2,5, 0,192 → 0,236), ce qui confirme la grande sensibilité au bruit de la dérivée seconde. Au-delà de $\sigma \approx 2$ à 2,5, la performance baisse pour tous : les contours fins disparaissent et les contours proches se confondent (figure 4).

Taille du noyau. Elle n'a presque pas d'effet sur Sobel (*P* entre 0,406 et 0,407 pour les noyaux 1 à 7), car le seuil est relatif au maximum et le flou joue déjà le rôle de lissage. Pour Laplace, un grand noyau améliore légèrement le résultat (0,214 avec le noyau 1, 0,236 avec le noyau 7), car il ajoute du lissage.

5.3 Exemples de résultats

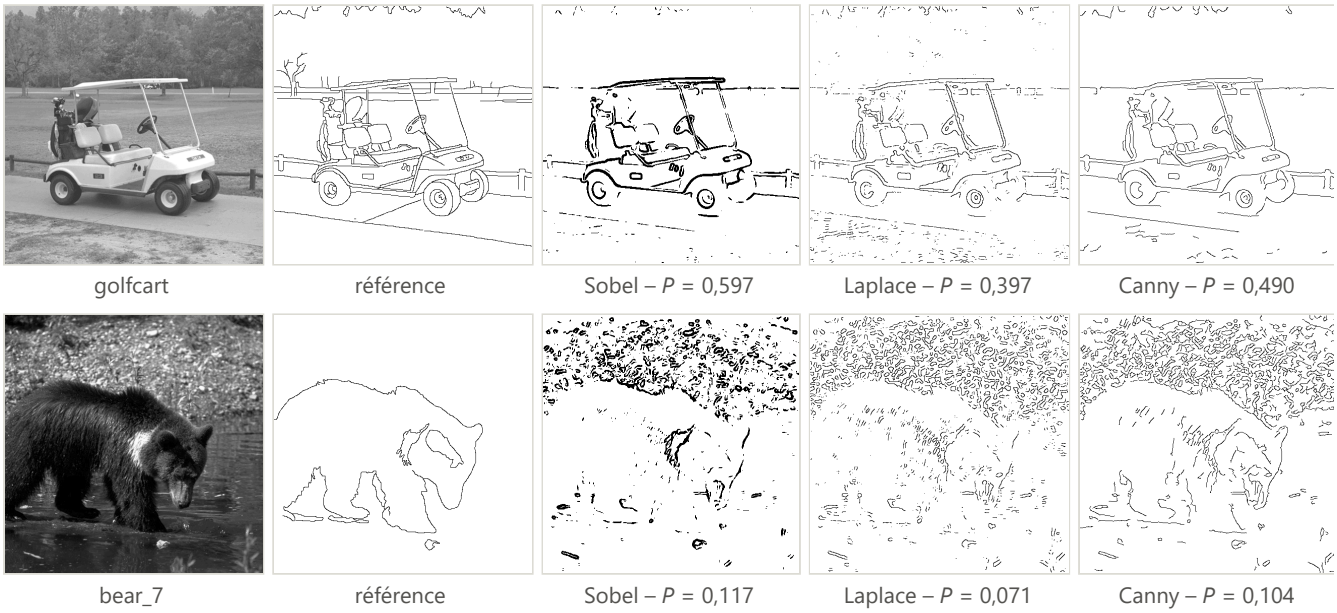
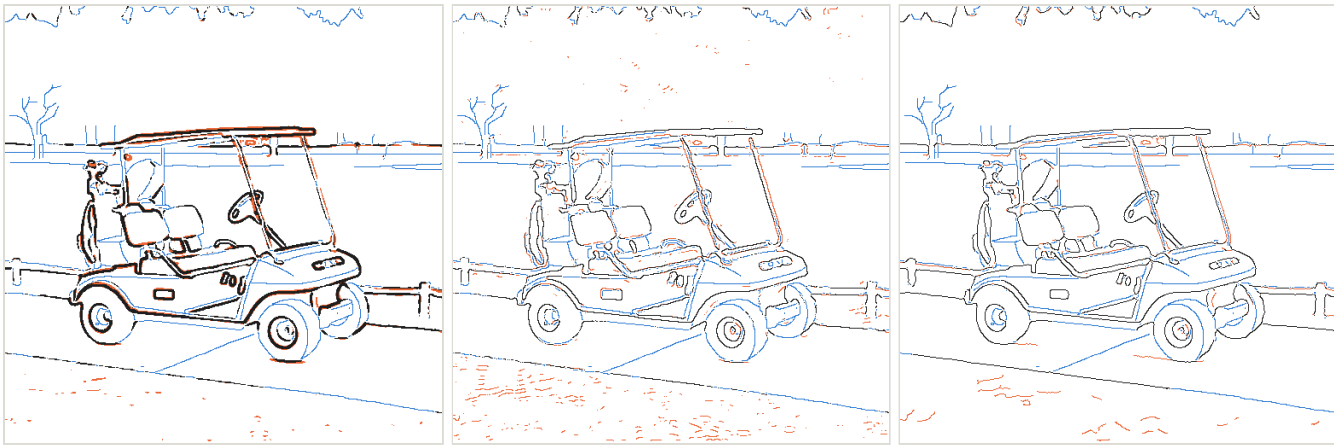


Figure 5 – Contours obtenus avec le réglage du tableau 1 sur une image favorable (golfcart, en haut) et sur l'image la plus difficile du lot (bear_7, en bas).



Sobel – *P* = 0,597, *TFP* = 0,159, *TFN* = 0,244 | Laplace – *P* = 0,397, *TFP* = 0,218, *TFN* = 0,385 | Canny – *P* = 0,490, *TFP* = 0,128, *TFN* = 0,382

Figure 6 – Comparaison avec la référence pour l'image golfcart : ■ pixel correct ■ faux positif ■ faux négatif (pixel de référence sans détection dans son voisinage).

5.4 Sensibilité à la méthode de comparaison

Détecteur	*P* selon le voisinage de tolérance			*P′* (3 × 3)
	exact (1 × 1)	3 × 3	5 × 5	
Sobel	0,148	0,407	0,528	0,260
Laplace	0,084	0,236	0,318	0,246
Canny	0,131	0,325	0,402	0,341

Tableau 3 – Performance moyenne de chaque détecteur (même réglage que le tableau 1) selon la méthode de comparaison. *P′* = (référence − *FN*) / ((référence − *FN*) + *FP* + *FN*) : chaque pixel de référence retrouvé n'est compté qu'une fois, quelle que soit l'épaisseur du contour détecté.

6. Interprétation et comparaison des trois détecteurs

6.1 Bonne détection

Canny est le détecteur qui manque le moins de contours tout en produisant le moins de faux contours. Son seuillage par hystérésis permet de garder les parties faibles d'un contour lorsqu'elles sont reliées à une partie forte, tout en rejetant les pixels faibles isolés. Sobel, avec un seul seuil, doit choisir entre perdre les contours faibles et garder la texture. Le Laplacien produit le plus de faux positifs (0,481 de taux moyen) : chaque petite variation d'intensité crée un passage par zéro, et même avec le seuil sur la pente, les zones texturées restent couvertes de petites boucles (figure 5, bear_7). Pour les trois détecteurs, la plupart des faux positifs sont de « vrais » contours d'intensité (herbe, feuillage, pelage) qui n'apparaissent pas dans la référence.

6.2 Bonne localisation

Canny donne des contours d'un pixel de large, centrés sur le maximum du gradient grâce à la suppression des non-maxima. Les passages par zéro du Laplacien font aussi un pixel de large, mais le fort lissage nécessaire déplace et arrondit les contours. Sobel ne fait que seuiller la norme du gradient : il produit des bandes de plusieurs pixels dont il ne garde pas le centre, ce qui donne la moins bonne localisation (figure 6). Le tableau 3 montre que la localisation exacte au pixel près est rare pour tous les détecteurs : en position exacte (1 × 1), la performance tombe à 0,148 (Sobel), 0,131 (Canny) et 0,084 (Laplace). Le tracé manuel de la référence n'est lui-même précis qu'à un ou deux pixels près, d'où l'intérêt du voisinage 3 × 3.

6.3 Meilleure méthode

Selon la mesure *P* demandée, Sobel est en tête (0,407), et il a le meilleur *P* sur 38 images sur 40. Mais cet avantage vient surtout de l'épaisseur de ses contours, c'est-à-dire d'une mauvaise localisation. En regardant les erreurs réellement commises, **Canny est le meilleur compromis** : moins de faux positifs, moins de faux négatifs, contours fins et continus. Il remplit les trois critères de la partie 2.4. Le Laplacien est le moins performant sur ces images naturelles. Il garde quelques atouts : il est isotrope, il utilise une seule convolution et il donne des contours fermés.

6.4 Variation en fonction des paramètres

Le réglage le plus important est le seuil, qui fixe le compromis entre faux positifs et faux négatifs ; vient ensuite le flou, surtout pour Canny et Laplace. Le seuil relatif de Sobel et de Laplace s'adapte automatiquement au contraste de chaque image, ce qui facilite le réglage. Les seuils absolus de Canny dépendent du contraste de l'image et du flou choisi. Dans tous les cas, l'optimum dépend de l'image : choisir le meilleur réglage image par image augmente *P* de 0,04 à 0,07 selon le détecteur, par rapport au réglage commun.

6.5 Méthode de comparaison : atouts et faiblesses

Atouts. La méthode est objective, automatique et reproductible. Elle donne un score unique par image, qui pénalise à la fois les faux positifs et les faux négatifs. Le voisinage 3 × 3 absorbe l'imprécision du tracé manuel : sans lui, presque aucun pixel ne serait « correct » (tableau 3).

Faiblesses.

- **Asymétrie du comptage.** Les pixels corrects sont comptés parmi les pixels détectés : un contour épais près d'une ligne de référence compte plusieurs fois. La méthode favorise donc les contours épais, c'est-à-dire une mauvaise localisation. Si chaque pixel de référence retrouvé n'est compté qu'une fois (*P'*, tableau 3), le classement s'inverse : Canny 0,341, Sobel 0,260, Laplace 0,246. Même en optimisant directement *P'*, Sobel ne dépasse pas 0,283.
- **Taille du voisinage arbitraire.** *P* fait plus que doubler entre la position exacte et le voisinage 3 × 3, puis augmente encore avec 5 × 5. Les valeurs absolues dépendent donc de ce choix ; seule la comparaison entre détecteurs avec le même voisinage a un sens.
- **Référence subjective.** Elle ne contient que les contours des objets, tracés par une personne. Les contours de texture, pourtant réels, comptent comme faux positifs. Cette base a été conçue pour évaluer des détecteurs capables de supprimer la texture ; les trois détecteurs locaux étudiés ici ne le sont pas, ce qui explique les valeurs de *P* assez faibles.
- **Pas de notion de continuité.** Un contour coupé en morceaux et un contour continu de même longueur obtiennent le même score, alors que le second est bien plus utile (pour la segmentation ou la transformée de Hough, par exemple).
- **Moyenne par image.** Toutes les images ont le même poids, quel que soit leur nombre de pixels de contour.

7. Conclusion

Le programme `homework3.cpp` permet de choisir un détecteur, de régler ses paramètres et de mesurer sa qualité sur un lot d'images. Sur la base de Groningen, avec le meilleur réglage commun de chaque détecteur, Sobel obtient la meilleure performance *P* (0,407), devant Canny (0,325) et Laplace (0,236). L'analyse des cinq grandeurs montre cependant que ce résultat vient de l'épaisseur des contours de Sobel, que la mesure récompense. Canny commet le moins d'erreurs, localise le mieux les contours et reste le meilleur compromis ; le Laplacien est le plus sensible au bruit. Pour les trois détecteurs, la principale source d'erreur est la texture, que des détecteurs locaux ne savent pas distinguer des contours d'objets. Pour aller plus loin, il faudrait une mesure qui compte les pixels corrects de façon symétrique, ou des seuils adaptés à chaque image.

Annexe – Mesures image par image

Image	Réf.	Sobel			Laplace			Canny		
		P	*TFP*	*TFN*	*P*	*TFP*	*TFN*	*P*	*TFP*	*TFN*
basket	6 938	**0,286**	0,604	0,110	0,103	0,781	0,117	0,240	0,460	0,300
bear	2 395	**0,395**	0,545	0,060	0,104	0,860	0,037	0,377	0,410	0,213
bear_2	5 978	**0,418**	0,279	0,303	0,243	0,385	0,372	0,300	0,171	0,529
bear_3	4 061	**0,281**	0,507	0,212	0,148	0,565	0,287	0,229	0,459	0,312
bear_4	2 863	**0,266**	0,666	0,068	0,109	0,812	0,078	0,149	0,786	0,065
bear_5	2 848	**0,319**	0,544	0,137	0,162	0,684	0,154	0,267	0,401	0,333
bear_6	4 304	**0,189**	0,661	0,150	0,098	0,710	0,192	0,146	0,642	0,212
bear_7	3 303	**0,117**	0,812	0,071	0,071	0,833	0,096	0,104	0,823	0,073
bear_8	3 012	**0,268**	0,480	0,252	0,083	0,761	0,156	0,135	0,697	0,168
bears	3 537	**0,364**	0,487	0,149	0,151	0,744	0,105	0,284	0,502	0,214
brush	8 675	**0,357**	0,279	0,364	0,207	0,427	0,366	0,229	0,107	0,664
buffalo	11 315	**0,581**	0,207	0,212	0,397	0,223	0,380	0,497	0,203	0,300
buffalo_2	6 781	**0,446**	0,299	0,255	0,289	0,392	0,319	0,344	0,368	0,288
elephant	11 052	**0,533**	0,135	0,332	0,378	0,130	0,492	0,451	0,098	0,451
elephant_2	4 613	**0,485**	0,116	0,399	0,342	0,302	0,356	0,359	0,038	0,602
elephant_3	3 291	**0,508**	0,140	0,352	0,347	0,086	0,568	0,439	0,066	0,495
elephant_4	7 848	**0,378**	0,190	0,432	0,295	0,290	0,414	0,371	0,246	0,384
elephants	6 225	**0,472**	0,232	0,296	0,432	0,218	0,350	0,446	0,157	0,397
elephants_2	7 566	0,316	0,050	0,634	0,205	0,082	0,713	**0,371**	0,115	0,514
gazelle	8 878	**0,393**	0,143	0,464	0,260	0,323	0,417	0,372	0,177	0,451
gazelle_2	6 632	**0,375**	0,175	0,450	0,216	0,334	0,449	0,300	0,356	0,344
gnu	6 056	**0,580**	0,350	0,070	0,382	0,452	0,166	0,500	0,354	0,147
gnu_2	5 064	**0,419**	0,269	0,312	0,226	0,541	0,233	0,343	0,230	0,428
goat	7 009	**0,440**	0,277	0,283	0,255	0,272	0,473	0,310	0,241	0,449
goat_2	6 360	0,316	0,208	0,477	0,215	0,477	0,308	**0,336**	0,399	0,265
goat_3	3 814	**0,423**	0,369	0,208	0,142	0,769	0,089	0,291	0,452	0,257
goat_4	4 326	**0,491**	0,358	0,151	0,270	0,487	0,243	0,420	0,248	0,332
goat_5	3 858	**0,310**	0,482	0,208	0,123	0,691	0,186	0,248	0,216	0,537
goat_6	3 050	**0,382**	0,445	0,174	0,124	0,785	0,092	0,187	0,720	0,093
goat_7	4 641	**0,451**	0,426	0,123	0,237	0,600	0,163	0,295	0,598	0,107
goat_8	5 552	**0,384**	0,341	0,275	0,228	0,510	0,262	0,326	0,478	0,196
golfcart	11 409	**0,597**	0,159	0,244	0,397	0,218	0,385	0,490	0,128	0,382
hyena	4 392	**0,628**	0,265	0,107	0,480	0,299	0,221	0,517	0,295	0,188
lions	9 005	**0,466**	0,336	0,198	0,271	0,407	0,322	0,343	0,303	0,354
rino	3 414	**0,437**	0,412	0,151	0,213	0,545	0,241	0,317	0,084	0,599
rino_2	5 694	**0,365**	0,536	0,099	0,204	0,633	0,163	0,274	0,569	0,157
rino_3	5 192	**0,431**	0,384	0,185	0,228	0,572	0,200	0,282	0,488	0,230
tiger	4 773	**0,424**	0,404	0,172	0,191	0,464	0,345	0,316	0,302	0,382
tire	9 653	**0,525**	0,237	0,237	0,294	0,224	0,481	0,432	0,122	0,446
turtle	4 220	**0,473**	0,227	0,300	0,311	0,336	0,353	0,352	0,262	0,385
moyenne		**0,407**	0,351	0,242	**0,236**	0,481	0,284	**0,325**	0,344	0,331

Tableau 4 – Mesures de chaque image avec le réglage du tableau 1 (Réf. = nombre de pixels de contour de la référence). En gras : le meilleur *P* de chaque image.